

Výstupy ŠVP

INF-08b V programu používám opakování (cykly) a seznamy. (Informatika 4, 2/2 · z INF-08)**INF-07a** Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešení problém ten nejvhodnější. (Informatika 4, 2/2 · z INF-07)**Učivo:** testování, optimalizace – vliv vstupních dat na spotřebované výpočetní zdroje

Připomenutí

$$f = \frac{n_A}{N}$$

$$\sigma = \sqrt{\frac{p(1-p)}{N}}$$

$$t \propto N$$

$$P(\text{aspoň jednou}) = 1 - (1-p)^n$$

$$\pi \approx 4 \cdot \frac{\text{zásahy}}{N}$$

Simulace náhody v Pythonu: hody kostkou a mincí, porovnání relativní četnosti s pravděpodobností, zákon velkých čísel a to, jak roste doba výpočtu s počtem opakování. Výsledky simulací se při každém spuštění liší – zapisuj je i s počtem pokusů. Podrobnosti najdeš v listu Must know.

A. Rozcvička

- Jaké hodnoty mohou vrátit: `random.randint(1, 6)`; `random.randrange(1, 6)`; `random.randint(0, 1)`; `random.random()`; `random.choice(["panna", "orel"])`; `random.randint(1, 6) + random.randint(1, 6)`? ★☆☆
- Urči pravděpodobnost při jednom hodu kostkou: padne 6; padne sudé číslo; padne číslo větší než 4; nepadne 1. ★☆☆
- Co program vypíše? Vysvětli, k čemu je `random.seed` dobrý. ★☆☆

```
import random
random.seed(7)
prvni = random.randint(1, 100)
random.seed(7)
druhe = random.randint(1, 100)
print(prvni == druhe)
```

- Napiš program, který hodí kostkou 20× a hody vypíše na jeden řádek oddělené mezerou. Kolik šestek očekáváš? ★☆☆
- Program má simulovat 6 000 hodů kostkou. Co vypíše? Najdi chybu. ★★☆☆

```
import random
cetnosti = [0] * 7
for i in range(6000):
    hod = random.randrange(1, 6)
    cetnosti[hod] += 1
print(cetnosti[1:])
```

- Po 600 hodech padla šestka 112×. Urči relativní četnost, očekávaný počet šestek a odchylku relativní četnosti od teorie. Je kostka podezřelá, když typická odchylka je při 600 hodech asi 0,015? ★★☆☆
- Proč se četnosti ukládají do seznamu `cetnosti = [0] * 7`, a ne `[0] * 6`? Jak by vypadal program se seznamem o 6 prvcích? ★☆☆
- Odhad: Hodíš 10 000×. Kolik šestek očekáváš? Typická odchylka počtu šestek je asi 37. Posuď výsledky 1 580, 1 700 a 2 000 šestek. ★★☆☆

B. Slovní úlohy

9. Spust' třikrát program, který simuluje 10 000 hodů kostkou a vypíše pro každé číslo četnost, relativní četnost a odchylku od $\frac{1}{6}$. Výsledky zapiš do tabulky. Mění se? Jak velké odchylky vycházejí? ★☆☆

```
import random
N = 10000
cetnosti = [0] * 7
for i in range(N):
    cetnosti[random.randint(1, 6)] += 1
for h in range(1, 7):
    f = cetnosti[h] / N
    print(h, cetnosti[h], round(f, 4), round(abs(f - 1 / 6), 4))
```

10. Zákon velkých čísel: Program simuluje hody pro $N = 10, 100, 1\,000, 10\,000, 100\,000$ a vypíše relativní četnost šestek a odchylku od teorie. Popiš, jak se odchylka mění, a porovnej s typickou odchylkou σ (Must know). ★★☆☆

```
import random
for N in (10, 100, 1000, 10000, 100000):
    sestky = 0
    for i in range(N):
        if random.randint(1, 6) == 6:
            sestky += 1
    f = sestky / N
    print(N, round(f, 4), round(abs(f - 1 / 6), 4))
```

11. Simuluj 36 000 hodů dvěma kostkami a spočítej četnosti součtů 2 až 12. Program vypíše i četnost přepočtenou na 36 hodů, aby šla porovnat s tabulkou v Must know. Který součet padá nejčastěji a proč? ★★☆☆

```
import random
N = 36000
soucty = [0] * 13
for i in range(N):
    soucty[random.randint(1, 6) + random.randint(1, 6)] += 1
for s in range(2, 13):
    print(s, soucty[s], round(soucty[s] / N * 36, 2))
```

12. Napiš funkci hod_do_sestky(), která hází kostkou, dokud nepadne šestka, a vrátí počet hodů. Zavolej ji 10 000× a spočítej průměrný počet hodů. Proč je zde vhodný cyklus while? ★★☆☆

```
import random
def hod_do_sestky():
    hody = 0
    while True:
        hody += 1
        if random.randint(1, 6) == 6:
            return hody

celkem = 0
for i in range(10000):
    celkem += hod_do_sestky()
print(celkem / 10000)
```

13. Změř, jak dlouho trvá simulace pro $N = 10^4, 10^5$ a 10^6 hodů. Vypočítej poměry časů. Odhadni, jak dlouho by trvalo 10^8 hodů. ★★☆☆

```
import random, time
for N in (10 ** 4, 10 ** 5, 10 ** 6):
    start = time.perf_counter()
    cetnosti = [0] * 7
    for i in range(N):
        cetnosti[random.randint(1, 6)] += 1
    konec = time.perf_counter()
    print(N, round(konec - start, 3), "s")
```

14. Paměť × čas: Verze A ukládá všechny hody (hody.append(h)) a nakonec spočítá hody.count(h) pro $h = 1$ až 6. Verze B hned zvyšuje počítadla. Porovnej, kolikrát každá verze projde data a kolik paměti potřebuje pro 10^6 hodů. Kdy je verze A přesto potřebná? ★★☆☆

15. Hráč hodí dvěma kostkami a vyhraje, když padne součet 7 nebo 11. Spočítej teoretickou pravděpodobnost výhry a ověř ji simulací 10 000 her. ★☆☆

```
import random
N = 10000
vyhry = 0
for i in range(N):
    s = random.randint(1, 6) + random.randint(1, 6)
    if s == 7 or s == 11:
        vyhry += 1
print(vyhry / N, 8 / 36)
```

16. Testování simulace (H47): Náhodný program nelze testovat porovnáním s pevným výstupem. Jak ho přesto otestovat? Vysvětli, co ověřuje tento test. ★★☆☆

```
import random
def simuluj(N):
    c = [0] * 7
    for i in range(N):
        c[random.randint(1, 6)] += 1
    return c
random.seed(1)
prvni = simuluj(1000)
random.seed(1)
druhy = simuluj(1000)
assert prvni == druhy and sum(prvni) == 1000
print("testy prošly")
```

C. Problémy a propojení

17. Monte Carlo: Náhodné body ve čtverci $[0; 1] \times [0; 1]$ padají do čtverťkuhu $x^2 + y^2 \leq 1$ s pravděpodobností $\frac{\pi}{4}$. Spuště program pro $N = 10^4$ a 10^6 a porovnej odhad s π . Typická chyba je pro 10^6 bodů asi 0,0016. Kolik bodů (a kolikrát víc času) by bylo třeba pro chybu 0,00005? ★★★

```
import random
N = 1000000
zasahy = 0
for i in range(N):
    x = random.random()
    y = random.random()
    if x * x + y * y <= 1:
        zasahy += 1
print(4 * zasahy / N)
```

18. Intuice × simulace: Myslíš, že ve 100 hodech mincí padne 6 stejných výsledků za sebou? Program 10 000× simuluje 100 hodů a vypíše průměrnou délku nejdelší série a podíl pokusů se sérií aspoň 6. Porovnej se svým odhadem. ★★★

```
import random
def nejdelssi_serie(n):
    predchozi, delka, nej = None, 0, 0
    for i in range(n):
        m = random.choice("P0")
        delka = delka + 1 if m == predchozi else 1
        nej = max(nej, delka)
        predchozi = m
    return nej

vysledky = [nejdelssi_serie(100) for i in range(10000)]
print(sum(vysledky) / 10000, sum(1 for v in vysledky if v >= 6) / 10000)
```

19. Matematika × informatika (H53): Hráč de Méré sázel, že ve 4 hodech kostkou padne aspoň jedna šestka. Spočítej pravděpodobnost výhry podle vzorce $1 - (1 - p)^n$ a ověř ji simulací. Proč se v simulaci hodí break?

★★☆

```
import random
N = 100000
uspech = 0
for i in range(N):
    for j in range(4):
        if random.randint(1, 6) == 6:
            uspech += 1
            break
print(uspech / N, 1 - (5 / 6) ** 4)
```

20. Narozeninový problém: Odhadni, jaká je pravděpodobnost, že ve třídě 30 žáků mají aspoň dva narozeniny ve stejný den (365 dní, stejně pravděpodobné). Ověř simulací a porovnej s výpočtem $1 - \frac{365 \cdot 364 \cdot \dots \cdot 336}{365^{30}}$. ★★☆

```
import random
def shoda(pocet):
    dny = []
    for i in range(pocet):
        d = random.randint(1, 365)
        if d in dny:
            return True
        dny.append(d)
    return False

N = 10000
print(sum(1 for i in range(N) if shoda(30)) / N)
```

Hotové úlohy k procvičení na internetu

- **Nauč se Python (PyLadies) – Základní funkce (modul random)** – vestavěné funkce a moduly math a random (randrange, uniform)
<https://naucse.python.cz/course/pyladies/beginners/basic-functions/>
- **Nauč se Python (PyLadies) – Seznamy** – seznamy a funkce random.choice a random.shuffle
<https://naucse.python.cz/course/pyladies/beginners/list/>
- **iMyšlení – Základy programování v jazyce Python pro SŠ** – učebnice schválená MŠMT s lekcí Náhoda a pracovními listy ke stažení
<https://archiv-imysleni.npi.cz/ucebnice/zaklady-programovani-v-jazyce-python-pro-stredni-skoly.html>

★ základní · ★★ střední · ★★★ náročnější. Vyber si, které úlohy spočítáš; řešení ukládej do svého portfolia.

Výsledky ke kontrole

- 1–6; 1–5; 0 nebo 1; desetinné číslo $0 \leq x < 1$; panna nebo orel; 2–12 (součty nejsou stejně pravděpodobné)
- $\frac{1}{6}$; $\frac{1}{2}$; $\frac{1}{3}$; $\frac{5}{6}$
- True – stejné semínko dá stejnou posloupnost „náhodných“ čísel; využití při ladění a testech (výsledek jde zopakovat)
- for i in range(20): print(random.randint(1, 6), end=" ");
očekávaně asi $\frac{20}{6} \doteq 3$ šestky, ale klidně 0 nebo 7
- např. [1172, 1188, 1284, 1192, 1164, 0] – šestka nepadne nikdy, čísla 1–5 asi 1 200×; randrange(1, 6) nezahrnuje 6 → randint(1, 6)
- $f \doteq 0,187$; očekávaně 100 šestek; odchylka $\doteq 0,020$, tj. asi $1,3\sigma$ – běžná náhodná odchylka, podezřelá není
- indexy 0–6, hod h jde přímo do cetnosti[h], index 0 se nevyužije; se 6 prvky: cetnosti[hod - 1] += 1
- očekávaně $\doteq 1\,667$; 1 700 zcela běžné (+33); 1 580 méně časté (–87, asi $2,3\sigma$), ale možné; 2 000 (+333, asi 9σ) prakticky vyloučené → chyba v programu nebo cinknutá kostka
- četnosti kolem 1 667, relativní četnosti kolem 0,167; při každém spuštění jiné hodnoty; odchylky řádově tisíce, jen výjimečně přes 0,01
- odchylka se s rostoucím N zmenšuje (nikoli pravidelně – je to náhoda); typické hodnoty σ : 0,118; 0,037; 0,012; 0,0037; 0,0012 – $10\times$ víc bodů zmenší odchylku asi $3\times$
- nejčastěji 7 (asi 6 000×, 6 z 36 dvojic), nejméně 2 a 12 (asi 1 000×); přepočtené hodnoty blízko 1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1
- průměr kolem 6 (teoreticky $\frac{1}{p} = 6$); počet hodů předem neznáme → while
- časy závisí na počítači (např. 0,007 s; 0,08 s; 0,7 s); poměr sousedních časů asi 10 → čas roste lineárně; 10^8 hodů asi $100\times$ déle než 10^6 , tj. zhruba minuta
- A: 1 průchod při házení + 6 průchodů count → asi $7N$ kroků, seznam s 10^6 prvky (≈ 8 MB jen pro odkazy); B: N kroků, 7 počítadel; A je potřebná, když záleží na pořadí hodů (série, grafy průběhu)
- $P = \frac{6+2}{36} = \frac{2}{9} \doteq 0,222$; simulace dá hodnotu blízko 0,22 ($\pm 0,01$)
- vypíše testy prošly; se stejným semínkem musí dvě simulace dát totéž a součet četností musí být N ; dál lze testovat vlastnosti (četnost nuly je 0, pro velké N relativní četnosti blízko $\frac{1}{6}$)
- pro 10^6 bodů např. 3,1427 (chyba řádově tisíce); chybu zmenšit $\frac{0,0016}{0,00005} \doteq 32\times \rightarrow$ potřeba asi $32^2 \doteq 1\,000\times$ víc bodů, tj. zhruba 10^9 bodů a $1\,000\times$ delší výpočet – Monte Carlo je jednoduché, ale pomalu přesné
- průměrná nejdelší série asi 7, série délky aspoň 6 nastane asi v 80 % pokusů – lidé série v náhodě podceňují (odhalí „vymyšlené“ řady hodů)
- $1 - \left(\frac{5}{6}\right)^4 \doteq 0,518$; simulace dá asi 0,52; break ukončí hody po první šestce – další hody už výsledek nezmění (úspora kroků)
- přesně $\doteq 0,706$; simulace asi 0,70–0,71 – většina lidí odhaduje mnohem méně; počet dvojic žáků je $\binom{30}{2} = 435$

Řešení krok za krokem

Úloha 17

- Program odhaduje $\pi \approx 4 \cdot \frac{\text{zásahy}}{N}$; pro 10^6 bodů vyjde např. 3,1427 (chyba řádově tisíce).
- Chyba Monte Carla klesá jako $\frac{1}{\sqrt{N}}$: $100\times$ víc bodů = $10\times$ menší chyba.
- Chybu je třeba zmenšit $\frac{0,0016}{0,00005} = 32\times$.
- Počet bodů proto vzroste $32^2 = 1\,024 \doteq 1\,000\times$, tj. na zhruba 10^9 bodů.
- Čas roste lineárně s počtem bodů → asi $1\,000\times$ delší výpočet; metoda je jednoduchá, ale pomalu přesná.

Úloha 18

- Odhad si zapiš předem (většina lidí tipuje nejdelší sérii 3–4).
- Funkce nejdelssi_serie počítá délku aktuální série a pamatuje si maximum; program ji zavolá $10\,000\times$ pro 100 hodů.
- Průměr seznamu vysledky vyjde asi 7, podíl pokusů se sérií aspoň 6 asi 0,8 (80 %).
- Závěr: v náhodě jsou dlouhé série běžné – kdo si hody vymýšlí, série podceňuje, a tak se vymyšlená řada pozná.